



THE FREE GUIDE · 2ND EDITION

B2B Software & IT Buyer's Guide

How to choose, scope, and budget custom software and IT services — with real pricing benchmarks and a live case study, so you avoid the expensive mistakes most businesses make on their first project.

INSIDE THIS GUIDE

- 01** Build vs. buy vs. customise
- 02** The real cost of custom software
- 03** 11 questions to ask any partner
- 04** Red flags that predict failure
- 05** Owning your software (no lock-in)
- 06** Real project, real numbers
- 07** Buying IT services the same way
- 08** An 8-step scoping checklist

INTRODUCTION

Buy software like it's an investment, not a purchase

Most businesses don't fail at software because the technology is hard. They fail because the decisions around it are made without a clear framework — the wrong build-vs-buy call, a budget set by guesswork, or a partner chosen on price alone. This guide gives you that framework: practical, jargon-free, and written from real B2B delivery experience — including real pricing benchmarks and a live project you can go and click through yourself.

1 Build vs. buy vs. customise

Three paths, and most teams pick the wrong one by default. Use this rule of thumb:

- **Buy off-the-shelf** when the process is generic and you're happy to adapt to the tool (e.g. email, accounting).
- **Customise a foundation** when 70% of what you need exists but the last 30% is your competitive edge.
- **Build custom** when the workflow *is* the business — the thing no off-the-shelf tool models well.

The real question

Don't ask "what software should we buy?" Ask "where does our process differ from everyone else's?" Buy the generic parts; build only the parts that make you different. Chapter 6 walks through a real example of a business that got this call right.

2 The real cost of custom software

The build is rarely the biggest cost — the total cost of ownership is. Budget for all five:

- **Discovery & design** — scoping the right thing before writing code (saves the most money).
- **Build** — the visible cost everyone quotes.
- **Integration & data** — connecting existing tools and migrating data is often underestimated.
- **Hosting & infrastructure** — cloud, backups, monitoring; ongoing, not one-off.
- **Maintenance & support** — budget 15–25% of build cost per year to keep it healthy.

CHAPTER 2, CONTINUED

A real ballpark: what these projects actually cost

Most guides tell you to "get quotes." That's fair, but it leaves you walking in blind. Here are indicative starting prices (INR) for common project types, so you have a number to sanity-check any quote against before you take a single call.

PROJECT TYPE	TYPICAL TIMELINE	INDICATIVE STARTING PRICE
Workflow automation	~5 weeks	₹80,000
API / system integration	~4 weeks	₹60,000
Web application	~7 weeks	₹1,10,000
Custom CRM platform	~8 weeks	₹1,20,000
Custom ERP system	~10 weeks	₹1,80,000
Mobile application	~12 weeks	₹2,00,000

Starting prices for standard complexity, one user role, no extra modules or integrations. Add roughly ₹15,000–18,000 per extra module or integration, and 30–65% for advanced or bespoke UI/logic. Ongoing maintenance and support typically runs 15–25% of build cost per year.

How to use this table

If a quote comes in far below these numbers with the same scope, ask what's being cut — usually discovery, documentation, or testing. If it's far above, ask what's driving the gap: more modules, more integrations, or a less efficient team.

Rule of thumb

If a quote only covers "the build," it's incomplete. A trustworthy partner shows you the ownership cost up front — even the parts that aren't theirs to bill.

3 11 questions to ask any software or IT partner

- Who owns the code and the IP when we're done?
- What does handover include — documentation, repo access, deployment?
- How do you scope and price: fixed bid, sprints, or time-and-materials?
- What happens if we want to leave or switch teams mid-project?
- Who maintains it after launch, and what does support cost?
- How do you handle security, access, and our data?
- Can we see a project you delivered in our industry or stack?
- **Can you show a live, currently-running deployment we can open ourselves** — not just screenshots or a sales deck?
- Who is actually on the team — seniors, or juniors behind a sales pitch?
- How will we see progress (demos, standups, environments)?
- What's your plan for the parts you *don't* build (hosting, integrations)?

Green flag

A good partner answers "who owns the code?" with "you do" — instantly, in writing. A great one also hands you a live URL to a project they built, right there in the call. Anything vaguer than that on either question is a lock-in or credibility risk.

“

Fair pricing and no lock-in. They scoped exactly what we needed and delivered on time.

Director · Retail Business

4 Red flags that predict a failed project

- A fixed price quoted before anyone understands the workflow.
- No discovery phase — straight to "we'll start building Monday."
- Vague ownership of code, accounts, or data.
- One salesperson, then silence — you never meet the builders.
- No staging environment or demos; you only see the end result.
- Pressure to sign fast with a discount that "expires Friday."
- No live, working reference you can go and click through yourself.

5 Owning your software: handover & no lock-in

You should be able to walk away with everything and keep running. Insist on:

- Source code in *your* repository, not the vendor's private one.
- Your own cloud, domain, and service accounts — in your name.
- Documentation a new developer could pick up cold.
- A clean deployment process (CI/CD), not a black box only they can run.

“

Clean code, clear documentation, and a proper handover. Our in-house team picked it up without any trouble.

Head of Engineering · Product Company

6 Real project, real numbers

Chapter 1 said: build custom when the workflow *is* the business. Here's what that looks like in practice — two live projects, not portfolio mockups.

Construction ERP V6

CONSTRUCTION / EPC

A multi-tenant, modular-monolith ERP that runs an EPC contractor end to end — projects and buildings, BOQ and delivery challans, drawing submission workflows, procurement planning, and full execution planning with WBS, dependencies, EVM, and AI forecasting.

- Real-time execution planning over WebSockets — WBS, Gantt, daily progress, and S-curve / EVM analytics.
- License-driven module enablement with company isolation and role-based access.
- Workflow Studio + runtime for configurable approval paths, plus Excel / Word / PDF report exports.

Live production deployment oeeppl.muskit.in →

React Vite FastAPI PostgreSQL WebSockets

AI HVAC Load & Design Platform

HVAC ENGINEERING

An AI-integrated HVAC load calculator and design studio that sizes heat loads and airflow, lays out diffusers, selects equipment, and estimates cost — then proposes cost-effective, balanced, and efficiency-first design concepts.

- AI Design Studio compares alternative concepts and turns warnings into specific corrective actions.
- ISO 14644 cleanroom mode (ISO 5–9) sizing recirculation airflow, filtration, and pressure regimes.
- Local rule-based engineering core always works; an optional AI layer enhances the guidance.

Live at hvac.muskit.in

Node.js JavaScript PostgreSQL OpenAI

“

Musk-IT replaced our spreadsheets with a proper system. Production, stock, and dispatch finally live in one place, and the team actually uses it.

Operations Head · Manufacturing SME

7 Buying IT services the same way

Everything so far has been about buying software. The same discipline applies when you're buying cloud, managed IT, cybersecurity, or IT consulting — the failure modes just look a little different.

Ask before you sign

- Is uptime and response time in writing, with a real SLA — not just "we'll get to it"?
- Are cloud, domain, and admin accounts registered in *your* name, or theirs?
- Do you get proactive monitoring and patching, or only reactive fixes when something breaks?
- Is there a costed roadmap tying IT spend to business outcomes, or just a list of tools?

Red flags here too

- No SLA, or one that's vague about response and resolution time.
- Your cloud/domain accounts sit under the vendor's organisation, not yours.
- Security is a one-time audit, not a continuous practice.
- No visibility into monitoring, backups, or incident history.

Rule of thumb

Whether it's a build or a managed service, the test is the same: can you walk away with what's yours — your code, your accounts, your data — and keep running without that vendor? If the answer is no, it's not a partnership, it's a dependency.

“

Integrations that used to be manual now just happen. Our tools finally talk to each other.

IT Manager · Distribution Company

8 Scope your first project in 8 steps

- ☐ 1. Write the problem in one sentence — the outcome, not the feature.
- ☐ 2. List the workflow steps that are manual or error-prone today.
- ☐ 3. Mark which steps are generic (buy) vs. your edge (build).
- ☐ 4. Name the systems it must connect to (CRM, ERP, payments, email) and any IT services (cloud, security) it depends on.
- ☐ 5. Decide who owns code, cloud, and data (answer: you).
- ☐ 6. Set a budget range including ownership cost, not just build — use Chapter 2's table as a starting point.
- ☐ 7. Agree how you'll see progress — demos and environments.
- ☐ 8. Define "done": handover, docs, deployment, and support terms.

+ How Musk-IT can help

Musk-IT is a B2B custom software and IT solutions provider. We build software around your real workflows — ERP, CRM, workflow automation, web & mobile apps, and API integrations — and run the IT behind it: cloud & infrastructure, managed IT & support, cybersecurity, and IT consulting. You own the code, with no lock-in — and you can see it running live before you sign anything.

Two free ways to start

1) Book a free consultation and we'll help you apply this guide to your project. 2) Claim a free IT & security audit — we'll review your cloud, security, and systems and hand you a prioritised action plan. Visit muskit.in to get started.